

Clashes @Biafra – Game Design Brief

Game Concept

Clashes @Biafra is a prototype third-person shooter game inspired by history and set during the 1967–1970 Nigerian Civil War. Drawing inspiration from historical African resistance—such as the Adwa case study examined by <https://rof.game/> detailing the Ethiopian Battle of Adwa against Italian colonial forces—the game casts the player as a solitary soldier defending a key jungle position against advancing enemy forces.

The visual essence of the prototype will focus on a war-torn, dense trench in the Eastern Nigeria (then Biafra) environment. For the scope of this sprint, the environment and characters will utilize undetailed 3D models to prioritize technical functionality and smooth mechanics over high-fidelity art.

Players must navigate the terrain, manage limited ammunition, and survive a skirmish by eliminating hostile patrols before their own health is depleted. The prototype emphasizes responsive third-person movement, tactical positioning, and core combat loops.

A solo developer is building this prototype during a 6-day sprint, focusing on delivering a technically functional, mechanically sound, and culturally resonant gameplay experience that meets all curriculum requirements.

Genre

- **Primary Genre:** Third-Person Shooter (TPS)
- **Secondary Genres:** Historical Action, Survival Prototype

Core Mechanics

The gameplay follows a focused combat loop:

1. **Navigate the Outpost:** Player uses a third-person controller to move through the environment, dynamically tracked by a follow camera.
2. **Resource Management:** Player monitors current ammunition and must trigger reloads or manage cooldowns during firefights.
3. **Engage Enemies:** Player aims via an on-screen crosshair and fires using a raycasting shooting system.
4. **Survive and Eliminate:** Players must avoid taking fatal damage from enemy AI while systematically hunting them down.
5. **Victory/Defeat:** Earn a "Win" screen by successfully eliminating all enemy AI in the level. Trigger a "Game Over" screen if the player's health drops to zero.

Supporting Systems & Technical Requirements

To meet the specific milestone objectives, the prototype will feature the following integrated systems:

- **Player & Camera System:** A robust third-person player controller paired with a

- Cinemachine FreeLook camera for smooth, AAA-style tracking.
- **Combat System:** Raycast-based shooting mechanics integrated with a constrained Ammo System (including reload logic).
- **Artificial Intelligence:** At least two (2) Enemy AI units utilizing Unity's NavMesh. The AI will feature a Finite State Machine with two core behaviors:
 - *Patrol:* Wandering between set waypoints.
 - *Chase:* Detecting the player and closing the distance to attack.
- **Health System:** Modular health scripts applied to both the Player and Enemy AI, allowing them to take damage and be destroyed.
- **User Interface (UI):** A HUD utilizing Unity Canvas to display:
 - Player Health (Visual Slider/Bar)
 - Ammo Counter (Text)
 - Aiming Crosshair (Center Screen Icon)
- **Game Flow System:** A Title Screen scene with a functional **Start Game** button, transitioning to the main gameplay scene, and concluding with a Win/Game-Over screen.
- **Audio-Visual Feedback (Juice):**
 - *Audio:* Looping background suspense music and at least 3 distinct Sound Effects (Gunshot, Death/Scream, and footsteps sounds).
 - *Particles:* At least 2 Particle Systems (A muzzle flash at the gun barrel when shooting, and a hit/blood effect when an enemy takes damage or is destroyed).

Target Platform

- **Primary Platform:** PC (Windows Executable)
- **Secondary Platform:** Web Browser (WebGL) - *Depending on build time optimization.*

Game Engine

- **Unity 3D Engine:** Unity was selected because it supports both Windows and WebGL deployment, offers native tools required for the prototype (Cinemachine, NavMesh, Particle System), and is the mandated engine for the Learnable Game Development curriculum sprint.

Project Organization

The project will maintain a highly organized folder structure (e.g., `_Scripts`, `_Prefabs`, `_Scenes`, `_Audio`) to ensure a professional, collaborative-ready workspace. All core game objects (Player, Enemy AI) will be saved as reusable Prefabs.